

Flutter Development Log

ES — Software Engineering

March 2, 2026

Contents

1	Environment Setup	2
1.1	Install Flutter SDK	2
1.2	Add Flutter to PATH	2
1.3	Install Android Studio	2
1.4	Verify the Installation	2
1.5	Install KVM/QEMU for Hardware Acceleration	3
1.6	Set Up the Android Emulator	3
1.7	Troubleshooting: Emulators Not Detected by Flutter	4
2	App Development	5
2.1	Create the Project Skeleton	5
2.2	Generate the App Code with ChatGPT	5
2.3	Replace the Default Code	6
2.4	Run the App	6
3	Critical Analysis	7

1 Environment Setup

Goal: Set up a Flutter development environment on Linux to build a simple mobile app.

1.1 Install Flutter SDK

Downloaded the Flutter SDK from the official website:

- [Download](#)

Created a dedicated folder in the home directory and extracted the SDK there using the GUI:

```
mkdir ~/develop
```

1.2 Add Flutter to PATH

To make the `flutter` command available system-wide, added Flutter to the PATH by editing the shell environment configuration file (`~/.zshenv`):

```
export PATH="$PATH:$HOME/develop/flutter/bin"
```

Reloaded the shell configuration:

```
source ~/.zshenv
```

1.3 Install Android Studio

Installed Android Studio via the AUR using an AUR helper:

```
yay -S android-studio
```

Launched Android Studio and completed the initial setup wizard, which installed the Android SDK and necessary build tools referenced by the following [guide](#).

1.4 Verify the Installation

Ran Flutter's built-in diagnostic tool to check that everything was correctly installed:

```
flutter doctor -v
```

Accepted the Android licenses when prompted:

```
flutter doctor --android-licenses
```

Re-ran `flutter doctor` until all required dependencies were marked as resolved.

1.5 Install KVM/QEMU for Hardware Acceleration

Hardware acceleration setup was performed following community documentation and the Arch Wiki on KVM.

In Linux, the Android emulator uses KVM for hardware acceleration. Before installing, verified that the CPU supports virtualisation with:

```
LC_ALL=C.UTF-8 lscpu | grep Virtualization
grep -E --color=auto 'vmx|svm|0xc0f' /proc/cpuinfo
zgrep CONFIG_KVM= /proc/config.gz
lsmod | grep kvm
lscpu | grep -i Virtualization
```

These commands confirm whether hardware virtualisation is supported and whether the KVM kernel module is already loaded.

Installed KVM/QEMU and related virtualisation tools:

```
sudo pacman -S qemu-full qemu-img libvirt virt-install virt-
  manager virt-viewer \
edk2-ovmf dnsmasq swtpm guestfs-tools libosinfo tuned
```

Enabled the libvirt service so it runs automatically on boot:

```
sudo systemctl enable libvirtd.service
```

Validated that the host is correctly configured for QEMU/KVM virtualisation:

```
sudo virt-host-validate qemu
```

Note: These steps are specific to Arch-based Linux distributions. Users on other distributions should refer to their package manager and the [Arch Wiki — KVM](#) for equivalent steps.

Resources used: This section was completed with the help of the [Arch Wiki — KVM](#) and [ChatGPT](#), which was used to clarify setup steps and troubleshoot configuration.

1.6 Set Up the Android Emulator

Followed the [documentation](#) in order to set an android emulator with enabled hardware acceleration. Launched the emulator from within Android Studio to verify it started correctly.

Also verified that Flutter could detect both the emulator and a connected physical device by running:

```
flutter devices
flutter emulators
```

1.7 Troubleshooting: Emulators Not Detected by Flutter

After setting up the Android emulator in Android Studio, running `flutter emulators` returned no devices. The issue was diagnosed and resolved with the help of Claude.

Diagnosis steps:

Confirmed flutter was “healthy”:

```
flutter doctor -v
```

The Android SDK was found correctly at `~/Android/Sdk`. Checked whether any AVDs were visible to the emulator binary directly:

```
~/Android/Sdk/emulator/emulator -list-avds
```

This returned empty output, confirming Flutter had no AVDs to list. However, 3 virtual devices had already been created in Android Studio. Searched the filesystem for any `.avd` folders:

```
find ~ -name "*.avd" -type d 2>/dev/null
```

This revealed the AVDs were stored in a non-default location:

```
/home/tusk/.config/.android/avd/Medium_Phone.avd
/home/tusk/.config/.android/avd/Flutter_Emu.avd
/home/tusk/.config/.android/avd/TestEmu.avd
```

The emulator binary looks for AVDs in `~/.android/avd/` by default, but Android Studio on this system was storing them in `~/.config/.android/avd/` instead.

Fix:

Added the following environment variable to `~/.zshenv` to point the emulator to the correct location:

```
export ANDROID_AVD_HOME=$HOME/.config/.android/avd
```

Reloaded the shell configuration and confirmed the fix:

```
source ~/.zshenv
~/Android/Sdk/emulator/emulator -list-avds
flutter emulators
```

All 3 virtual devices were now detected correctly.

Resources used: Diagnosed and resolved with the help of [Claude](#).

2 App Development

Goal: Create a simple Flutter app that displays “Hi! My name is Pedro” centered on the screen.

2.1 Create the Project Skeleton

Created a new Flutter project using the Flutter CLI, which automatically generates the project structure:

```
flutter create my_first_app
cd my_first_app
```

This generates the following structure, among others:

```
my_first_app/
|-- lib/
|   |-- main.dart    <- Main app code
|-- android/
|-- ios/
|-- pubspec.yaml     <- Dependencies configuration
|-- README.md
```

Opened the project in Android Studio to edit the code.

2.2 Generate the App Code with ChatGPT

Instead of writing the code manually, used ChatGPT to generate the app code.

Prompt sent to ChatGPT:

I'm a beginner learning Flutter. I want to create a very simple Flutter app from scratch that just displays the text “Hi! My name is Pedro” centered on the screen. Please give me the full code and explain each part briefly.

Output received:

ChatGPT provided the following code for `lib/main.dart` which was reviewed and matched expectations.

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
```

```
    return const MaterialApp(  
      home: Scaffold(  
        body: Center(  
          child: Text(  
            'Hi! My name is Pedro',  
            style: TextStyle(  
              fontSize: 24,  
            ),  
          ),  
        ),  
      ),  
    );  
  }  
}
```

Along with a brief explanation of each part:

- `import 'package:flutter/material.dart'` — imports Flutter's Material Design widget toolbox.
- `main()` — the entry point of every Dart/Flutter app; `runApp()` tells Flutter which widget to display first.
- `MyApp` extends `StatelessWidget` — a widget whose UI does not change dynamically, suitable for simple apps.
- `build()` — every widget must implement this method, which describes what the UI looks like.
- `MaterialApp` — the root of a Material Design app, sets up the basic app structure.
- `Scaffold` — provides the basic visual layout (background, body, optional app bar).
- `Center` — centers its child widget on the screen.
- `Text` — displays a string of text, styled with `TextStyle`.

2.3 Replace the Default Code

Replaced the contents of `lib/main.dart` with the code generated by ChatGPT.

2.4 Run the App

Ran the app on the emulator and on a physical device:

```
flutter run
```

Flutter detected the available devices automatically. The app launched and displayed “Hi! My name is Pedro” centered on the screen as expected.

3 Critical Analysis

Overall, the process of going from zero to a working Flutter app was manageable, though not entirely frictionless. The estimated time spent was between 4 and 6 hours, which aligns with the higher end of the assignment's estimated effort range.

The most straightforward part was the app development itself. ChatGPT generated working code on the first attempt, along with explanations of each part of the code. For a beginner, this was particularly valuable — not only did it produce a result instantly, but it also helped build an understanding of the Flutter widget tree and how a basic app is structured. Without GenAI assistance, writing even a simple app like this (ignoring the pre-made template generated when 'flutter create' is ran) would have required significantly more time spent reading documentation and learning the basics of Dart.

The setup process, on the other hand, required more effort. Installing Flutter, Android Studio, and KVM/QEMU on an Arch-based Linux system involved several steps that are not immediately obvious by looking at the base documentation and required a lot more research.

The most notable setbacks were the emulator detection issue, where Android Studio was storing AVDs in a non-default location (`~/.config/.android/avd/`) that the emulator binary was not checking, and the non-obvious way of checking for the system's ability of virtualization and setting up hardware acceleration. Claude was crucial in guiding both of these situations.

In retrospect, the Linux environment added friction that would likely not exist on Windows or macOS, where the default paths and tooling tend to be more standardised. That said, the issues encountered were solvable and the troubleshooting process itself was a valuable learning experience.

The video deliverable was recorded with the help of OBS-Studio and the screen sharing of my physical mobile device was made possible with the use of the 'scrcpy' package.